

자바 개발자를 위한 Kotlin v2.2.0

Control Flow

<https://kotlinlang.org/docs/home.html> 공식 문서를 기반으로 작성함

2025-07 조수연

조건문

if

<https://kotlinlang.org/docs/control-flow.html#if-expression>

- if-expression
 - Kotlin에선 if 문을 expression으로 사용할 수 있다.
 - 단, expression으로 사용할 경우 할당이 반드시 일어나야 하므로 else 문을 이용해 모든 경우를 처리해야 한다.

```
var max = a
if (a < b) max = b

// With else
if (a > b) {
    max = a
} else {
    max = b
}

// As expression
max = if (a > b) a else b

// You can also use `else if` in expressions:
val maxLimit = 1
val maxOrLimit = if (maxLimit > a) maxLimit else if (a > b) a else b

println("max is $max")
// max is 3
println("maxOrLimit is $maxOrLimit")
// maxOrLimit is 3
```


if

<https://kotlinlang.org/docs/control-flow.html#if-expression>

- if-expression
 - if 문의 마지막 expression이 if 문의 결과 값이 된다.

```
val max = if (a > b) {  
    print("Choose a")  
    a  
} else {  
    print("Choose b")  
    b  
}
```

when

<https://kotlinlang.org/docs/control-flow.html#when-expressions-and-statements>

- switch 문과 유사하지만 더 많은 기능을 제공하는 조건문
- statement 로도 사용 가능하고 expression으로도 사용할 수 있다.

```
val number = readln().toInt()
when(number % 2){
    0 -> println("even")
    1 -> println("odd")
}
```

```
val number = readln().toInt()
val message = when(number % 2){
    0 -> "even"
    else -> "odd"
}
println(message)
```

expression으로 사용할 경우
반드시 모든 경우를 다뤄야 하므로
else를 꼭 사용해야 한다.
단, enum 의 경우 모든 경우를 다루면 else 생략 가능

when

<https://kotlinlang.org/docs/control-flow.html#when-expressions-and-statements>

- 두 가지 이상의 조건을 한 번에 처리 가능

```
val type = readln()
val value = readln()
val input = when(type){
    "Int", "Long" -> value.toLong()
    "Double", "Float" -> value.toDouble()
    else -> 0
}
```

- in 및 is 연산자 사용 가능

```
val array = arrayOf(1, 3, 5, 7, 9)
val input = readln().toInt()
when(input){
    in array -> println("in array")
    in 10..20 -> println("in range")
    else -> println("invalid input")
}
```

```
fun hasPrefix(x: Any) = when(x) {
    is String -> x.startsWith("prefix")
    else -> false
}
```


반복문

for

<https://kotlinlang.org/docs/control-flow.html#for-loops>

- Iterator를 제공하는 객체를 iteration. for-each 구문과 같이 동작한다.

```
for (item in collection) print(item)
```

- 특정 숫자 범위를 이용하려면 Range를 사용할 수 있다.

```
for (i in 1..3) {  
    print(i)  
}  
for (i in 6 downTo 0 step 2) {  
    print(i)  
}
```


for

<https://kotlinlang.org/docs/control-flow.html#for-loops>

- index 도 필요한 경우 다음과 같이 사용할 수 있다.

```
for (i in array.indices) {  
    print(array[i])  
}
```

```
for ((index, value) in array.withIndex()) {  
    println("the element at $index is $value")  
}
```

while, do-while

- 기본적으로 Java와 용법이 같다.
- do-while의 경우 do 블록 안에서 선언한 변수를 while에서 사용할 수 있다.

```
while (x > 0) {  
    x--  
}  
  
do {  
    val y = retrieveData()  
} while (y != null) // y is visible here!
```

Return and jumps

Return and jumps

<https://kotlinlang.org/docs/returns.html>

- return
 - 자신을 감싼 가장 가까운 함수 또는 익명 함수를 반환한다.
- break
 - 자신을 감싼 가장 가까운 반복분을 종료한다.
- continue
 - 자신을 감싼 가장 가까운 반복문을 다음 단계로 진행시킨다.

Label

<https://kotlinlang.org/docs/returns.html>

- label
 - 이름@ 의 형태의 식별자.

```
loop@ for (i in 1..100) {  
    // ...  
}
```

- 반복문에 정의한 label을 이용해 중첩된 반복문에 break, continue를 적용할 수 있다.

```
loop@ for (i in 1..100) {  
    for (j in 1..100) {  
        if (...) break@loop  
    }  
}
```

Return

<https://kotlinlang.org/docs/returns.html>

- 주의할 점 - 람다식(Lambda expression)은 함수가 아니다.

```
fun main() {  
    foo()  
}  
  
fun foo(){  
    arrayOf(1, 2, 3, 4, 5).forEach {  
        if(it==3) return  
    }  
    println("dead code")  
}
```

forEach 의 파라미터로 넘어간 것은 람다식
return 을 감싸고 있는 가장 가까운 함수는 foo
return 하는 순간 foo 가 return 된다.

```
fun main() {  
    foo()  
}  
  
fun foo(){  
    arrayOf(1, 2, 3, 4, 5).forEach loop@{  
        if(it==3) return@loop  
    }  
    println("here")  
}
```

람다식을 label 로 지정할 경우 람다식만 종료할 수 있다.

Exceptions

Exceptions

<https://kotlinlang.org/docs/exceptions.html>

- Exception을 생성하기

```
val cause = IllegalStateException("Original cause: illegal state")

// Throws an IllegalArgumentException if userInput is negative
// Additionally, it shows the original cause, represented by the cause IllegalStateException
if (userInput < 0) {
    throw IllegalArgumentException("Input must be non-negative", cause)
}
```


Exceptions

<https://kotlinlang.org/docs/exceptions.html>

- Precondition 함수로 Exception 발생시키기

Precondition function	Use case	Exception thrown
<code>require()</code>	Checks user input validity	<code>IllegalArgumentException</code> ↗
<code>check()</code>	Checks object or variable state validity	<code>IllegalStateException</code> ↗
<code>error()</code>	Indicates an illegal state or condition	<code>IllegalStateException</code> ↗

```
fun getIndices(count: Int): List<Int> {  
    require(count >= 0) { "Count must be non-negative. You set count to $count." }  
    return List(count) { it + 1 }  
}
```

```
fun getValue(someState:String): String {  
    val state = checkNotNull(someState) { "State must be set beforehand!" }  
    check(state.isNotEmpty()) { "State must be non-empty!" }  
    return state  
}
```

Exceptions

<https://kotlinlang.org/docs/exceptions.html>

- try-catch

```
try{  
    val number = readln().toInt()  
}catch (e: Exception){  
    println(e.message)  
}
```

```
try{  
    val number = readln().toInt()  
}catch (e: Exception){  
    println(e.message)  
}finally {  
    println("finished")  
}
```

- Kotlin 의 try-catch 문은 expression 이며 try 블록 또는 catch 블록의 값이 된다. finally 블록은 결과 값에 영향을 끼치지 않는다.

```
val number = try { readln().toInt() } catch (e: Exception) { 0 }
```